

Assignment #2 — Domain Model

QuickBite Food Delivery System

Group 7: Elephant-Yankee | Mike-Alfa | Alfa-Kilo | Sierra-Lima

1. System Functionalities

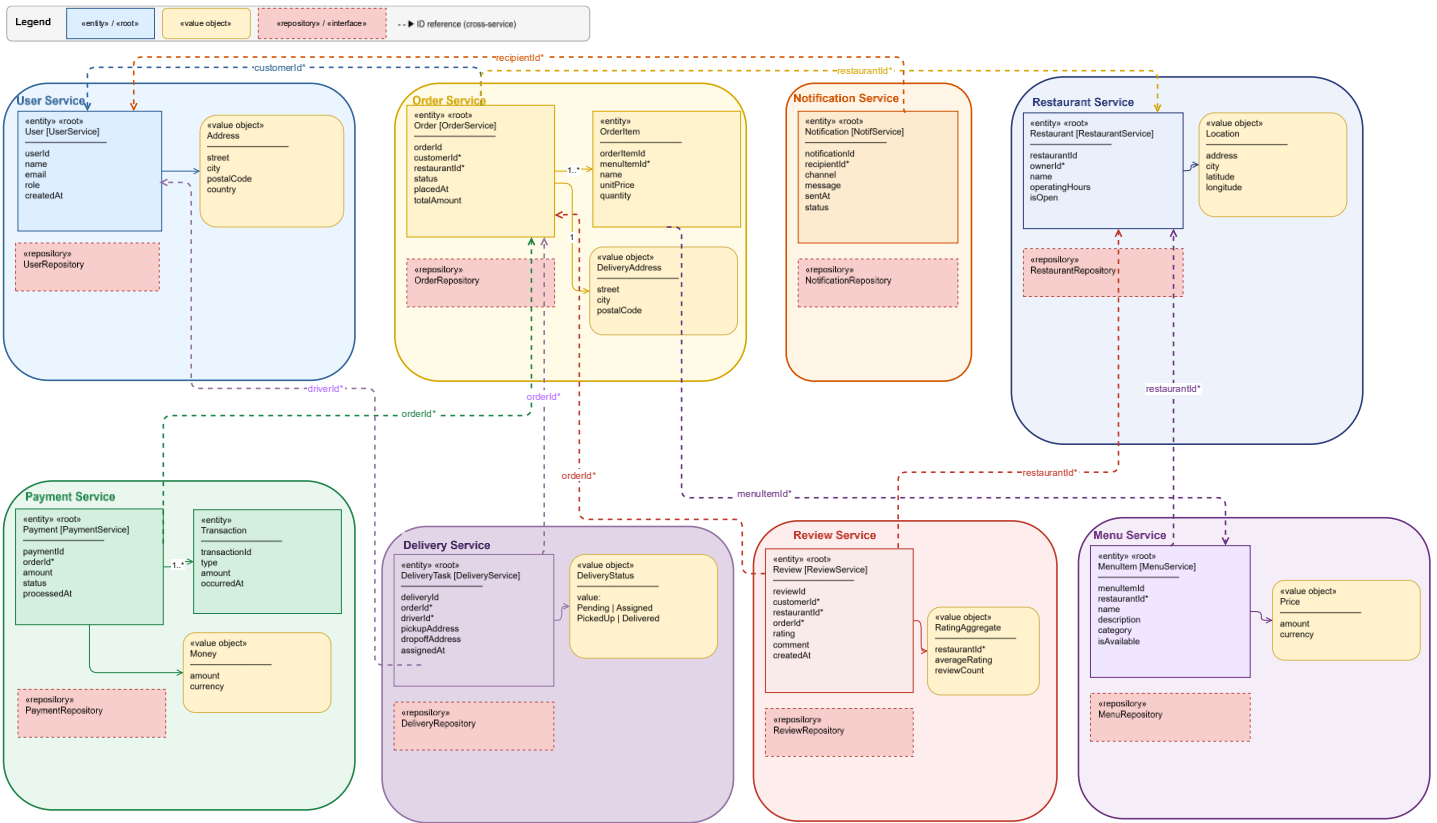
The following requirements describe the main operations supported by the QuickBite microservices, as derived from the architecture defined in Assignment 1.

ID	Service	Requirement
R1	User Service	The system shall allow a person to create an account as a customer, driver, or restaurant partner.
R2	User Service	The system shall allow a user to sign in to their account.
R3	User Service	The system shall allow a user to update their profile information and delivery addresses.
R4	Order Service	The system shall allow a customer to place an order by selecting a restaurant, menu items, and a delivery address.
R5	Order Service	When placing an order, the system shall validate the customer, confirm the restaurant is open, and verify item prices.
R6	Order Service	The system shall allow a customer to cancel a pending order before it is accepted.
R7	Order Service	The system shall update the order status when payment and delivery progress change.
R8	Payment Service	The system shall allow an order to initiate a payment charge for the total order amount.
R9	Payment Service	When a payment is processed, the system shall record the result and update the related order.
R10	Payment Service	The system shall allow a refund to be issued for a cancelled or failed order.
R11	Delivery Service	The system shall create a delivery task after payment for an order is confirmed.
R12	Delivery Service	The system shall assign an available driver to a delivery task.
R13	Delivery Service	The system shall allow a driver to update the delivery status (Assigned, PickedUp, Delivered).
R14	Delivery Service	When delivery progress changes, the system shall update the related order and inform the relevant user.
R15	Notification Service	The system shall send a notification to the relevant user when an order status changes.
R16	Notification Service	The system shall send notifications to users through supported communication methods.
R17	Review Service	The system shall allow a customer to submit a review for a completed order only after delivery is confirmed.
R18	Review Service	The system shall store and aggregate ratings per restaurant.
R19	Restaurant Service	The system shall allow a restaurant partner to register and manage their restaurant profile.

ID	Service	Requirement
R20	Restaurant Service	The system shall allow a restaurant to update its open/closed status and operating hours.
R21	Menu Service	The system shall allow a restaurant to add, update, and remove menu items with prices and availability.
R22	Menu Service	The system shall allow customers to browse menu items for a given restaurant.

2. Domain Model Diagram

The diagram below represents the full business-level domain model for QuickBite. It includes all 8 microservices, their domain concepts, DDD tactical elements (entities, value objects, aggregate roots, and repositories), and cross-service relationships via ID references.



3. Domain Concepts - DDD Reference

The table below documents all domain concepts across the 8 services, their DDD classification, attributes, and aggregate membership. Attributes marked with * are cross-service ID references. They reference the identity of a concept owned by another service without redefining it.

Service	Concept	Kind	DDD Role	Attributes
User Service	User	Entity	Aggregate Root	userId, name, email, role, createdAt
User Service	Address	Value Object	–	street, city, postalCode, country
User Service	UserRepository	Repository	Interface	
Order Service	Order	Entity	Aggregate Root	orderId, customerId*, restaurantId*, status, placedAt, totalAmount
Order Service	OrderItem	Entity	–	orderItemId, menuItemId*, name, unitPrice, quantity
Order Service	DeliveryAddress	Value Object	–	street, city, postalCode
Order Service	OrderRepository	Repository	Interface	
Payment Service	Payment	Entity	Aggregate Root	paymentId, orderId*, amount, status, processedAt
Payment Service	Transaction	Entity	–	transactionId, type (charge/refund), amount, occurredAt
Payment Service	Money	Value Object	–	amount, currency
Payment Service	PaymentRepository	Repository	Interface	
Delivery Service	DeliveryTask	Entity	Aggregate Root	deliveryId, orderId*, driverId*, pickupAddress, dropoffAddress, status: DeliveryStatus, assignedAt
Delivery Service	DeliveryStatus	Value Object	–	value: Pending Assigned PickedUp Delivered
Delivery Service	DeliveryRepository	Repository	Interface	
Notification Service	Notification	Entity	Aggregate Root	notificationId, recipientId*, channel, message, sentAt, status
Notification Service	NotificationRepository	Repository	Interface	
Review Service	Review	Entity	Aggregate Root	reviewId, customerId*, restaurantId*, orderId*, rating, comment, createdAt
Review Service	RatingAggregate	Value Object	–	restaurantId*, averageRating, reviewCount
Review Service	ReviewRepository	Repository	Interface	
Restaurant Service	Restaurant	Entity	Aggregate Root	restaurantId, ownerId*, name, location: Location, operatingHours, isOpen
Restaurant Service	Location	Value Object	–	address, city, latitude, longitude
Restaurant Service	RestaurantRepository	Repository	Interface	

Service	Concept	Kind	DDD Role	Attributes
Menu Service	MenuItem	Entity	Aggregate Root	menuItemid, restaurantId*, name, description, price: Price, category, isAvailable
Menu Service	Price	Value Object	–	amount, currency
Menu Service	MenuRepository	Repository	Interface	

4. Cross-Service Relationships

All cross-service relationships use ID references. A concept belonging to one service does not redefine or embed concepts from another service. It only holds the identifier. This preserves service autonomy while keeping the model coherent.

Relationship	Mechanism	Business Reason
Order → User	Order.customerId references User.userId	An order must identify which customer placed it.
Order → Restaurant	Order.restaurantId references Restaurant.restaurantId	An order must identify which restaurant it belongs to.
OrderItem → MenuItem	OrderItem.menuItemId references MenuItem.menuItemId	An order item must identify which menu item was ordered.
Payment → Order	Payment.orderId references Order.orderId	A payment is made for one specific order.
DeliveryTask → Order	DeliveryTask.orderId references Order.orderId	A delivery task is created for one specific order.
DeliveryTask → User (driver)	DeliveryTask.driverId references User.userId	A delivery task must identify which driver is assigned to it.
Review → Order	Review.orderId references Order.orderId	A review is linked to the order it is based on.
Review → Restaurant	Review.restaurantId references Restaurant.restaurantId	A review must identify which restaurant is being reviewed.
Review → User	Review.customerId references User.userId	A review must identify which customer submitted it.
Restaurant → User (owner)	Restaurant.ownerId references User.userId	A restaurant profile must identify its owner.
MenuItem → Restaurant	MenuItem.restaurantId references Restaurant.restaurantId	A menu item belongs to one restaurant.
RatingAggregate → Restaurant	RatingAggregate.restaurantId references Restaurant.restaurantId	A rating summary is maintained for one restaurant.
Notification → User	Notification.recipientId references User.userId	A notification must identify which user will receive it.

5. Individual Contributions

Each team member is responsible for two microservices as assigned in Assignment 1. The domain concepts, attributes, DDD elements, and cross-service relationships for each member's services are documented in the tables above.

Member	Service 1	Service 2	Cross-service relationship
Alfa-Kilo	User Service	Order Service	Order → User (customerId*), Order → Restaurant (restaurantId*)
Elephant-Yankee	Payment Service	Delivery Service	Payment → Order (orderId*), DeliveryTask → User (driverId*)
Mike-Alfa	Notification Service	Review Service	Notification → User (recipientId*), Review → Order (orderId*)
Sierra-Lima	Restaurant Service	Menu Service	Restaurant → User (ownerId*), MenuItem → Restaurant (restaurantId*)